SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA04**

COURSE OUTCOME COVERED

CO2: Apply data pre-processing and visualization techniques using Python libraries for effective data handling. (BL4)

QUESTIONS: 05 **Total Marks:100**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

IM.Gayathri(192472074)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.Gayathri

Annexure A

Analytical Problem Solving

1. Online Food Delivery Data Analysis
An online food delivery company wants to analyze order data to improve delivery performance and customer satisfaction. Explain how:
A. **Pandas** is used to read, clean, and filter order data
B. **Data preprocessing** is applied to handle missing delivery time, duplicate orders, and incorrect prices
C. **Matplotlib or Seaborn** is used to visualize delivery delays, top restaurants, and customer ratings

2. Bank Loan Approval Analysis
A bank wants to study customer loan applications and identify patterns related to loan approval and rejection. Explain how:
A. **Pandas DataFrame operations** are used to select income, loan amount, credit score, and approval status
B. **Preprocessing techniques** are used to handle missing income values, encode gender/employment type, and normalize loan amount
C. **Visualization** is used to compare approved and rejected applications based on income, credit score, and loan amount

3. Smart Energy Consumption Monitoring
A smart home system collects daily electricity consumption data from different appliances such as AC, fan, light, and refrigerator. Explain how:
A. **Pandas** is used to group electricity usage by appliance and date
B. **Data preprocessing** is used to remove abnormal readings, fill missing values, and calculate total power consumption
C. **Line charts and bar charts** are used to identify high-energy appliances and daily usage trends

4. Online Learning Platform Performance Analysis
An online learning platform wants to analyze student activity such as login time, video completion, quiz score, and assignment submission. Explain how:
A. **Pandas** is used to sort, filter, and rank students based on quiz scores and completion percentage
B. **Preprocessing** is applied to handle missing quiz scores, convert date formats, and create new columns such as performance category
C. **Seaborn and Matplotlib** are used to visualize student engagement, score distribution, and course completion status

5. Public Transport Passenger Data Analysis
A city transport department wants to analyze bus passenger data to improve route planning and reduce overcrowding. Explain how:
A. **Pandas** is used to analyze passenger count by route, stop, and time slot
B. **Data preprocessing** is used to clean invalid route numbers, handle missing passenger counts, and remove duplicate records
C. **Visualization techniques** are used to show peak travel hours, busiest routes, and passenger flow patterns

1. Online Food Delivery Data Analysis

A. Using Pandas to read, clean, and filter order data

Pandas is a Python library used to handle and analyze data. It reads order data from CSV files using `pd.read_csv()`. Data can be filtered based on delivery time, restaurant name, and customer ratings.

code:

```
import pandas as pd

df = pd.read_csv("food_orders.csv")
print(df.head())

# Filter orders with delivery time greater than 30 minutes
print(df[df["Delivery_Time"] > 30])
```

```
= RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DSA0402-FODS/ASSESSMEN
TS/C02-AT3/DATA_ANALYSIS.py
   Order_ID Restaurant  Delivery_Time  Price  Rating
0       101    Dominos           25.0    350       5
1       102        KFC           40.0    500       4
2       103     Subway           30.0    250       4
3       104    Dominos           55.0    400       3
4       105  Pizza_Hut           20.0    600       5
    Order_ID Restaurant  Delivery_Time  Price  Rating
1        102        KFC           40.0    500       4
3        104    Dominos           55.0    400       3
6        107     Subway           35.0    200       3
8        109  Pizza_Hut           45.0    700       2
9        110        KFC           32.0    550       4
11       112    Dominos           60.0    450       2
12       113  Pizza_Hut           38.0    650       4
16       117        KFC           42.0    500       3
17       118  Pizza_Hut           33.0    600       4
19       120    Dominos           50.0    400       3
```

B. Data preprocessing

- Missing values: Fill missing delivery times using the median.
- Duplicate orders: Remove repeated records using `drop_duplicates()`.
- Incorrect prices: Remove records with zero or negative prices.

code:

```
df["Delivery_Time"] = df["Delivery_Time"].fillna(
    df["Delivery_Time"].median()
)
df = df.drop_duplicates()
df = df[df["Price"] > 0]
```

```
B. CLEANED FOOD DELIVERY DATA
    Order_ID Restaurant  Delivery_Time  Price  Rating
0        101    Dominos           25.0    350       5
1        102        KFC           40.0    500       4
2        103     Subway           30.0    250       4
3        104    Dominos           55.0    400       3
4        105  Pizza_Hut           20.0    600       5
5        106        KFC           32.0    450       4
6        107     Subway           35.0    200       3
7        108    Dominos           28.0    350       5
8        109  Pizza_Hut           45.0    700       2
9        110        KFC           32.0    550       4
10       111     Subway           25.0    300       5
11       112    Dominos           60.0    450       2
12       113  Pizza_Hut           38.0    650       4
14       115     Subway           30.0    250       4
15       116    Dominos           25.0    350       5
16       117        KFC           42.0    500       3
17       118  Pizza_Hut           33.0    600       4
18       119     Subway           27.0    220       5
19       120    Dominos           50.0    400       3

Analysis completed successfully!
```

C. Data visualization

Matplotlib and Seaborn are used to visualize delivery delays, popular restaurants, and customer ratings.

code:

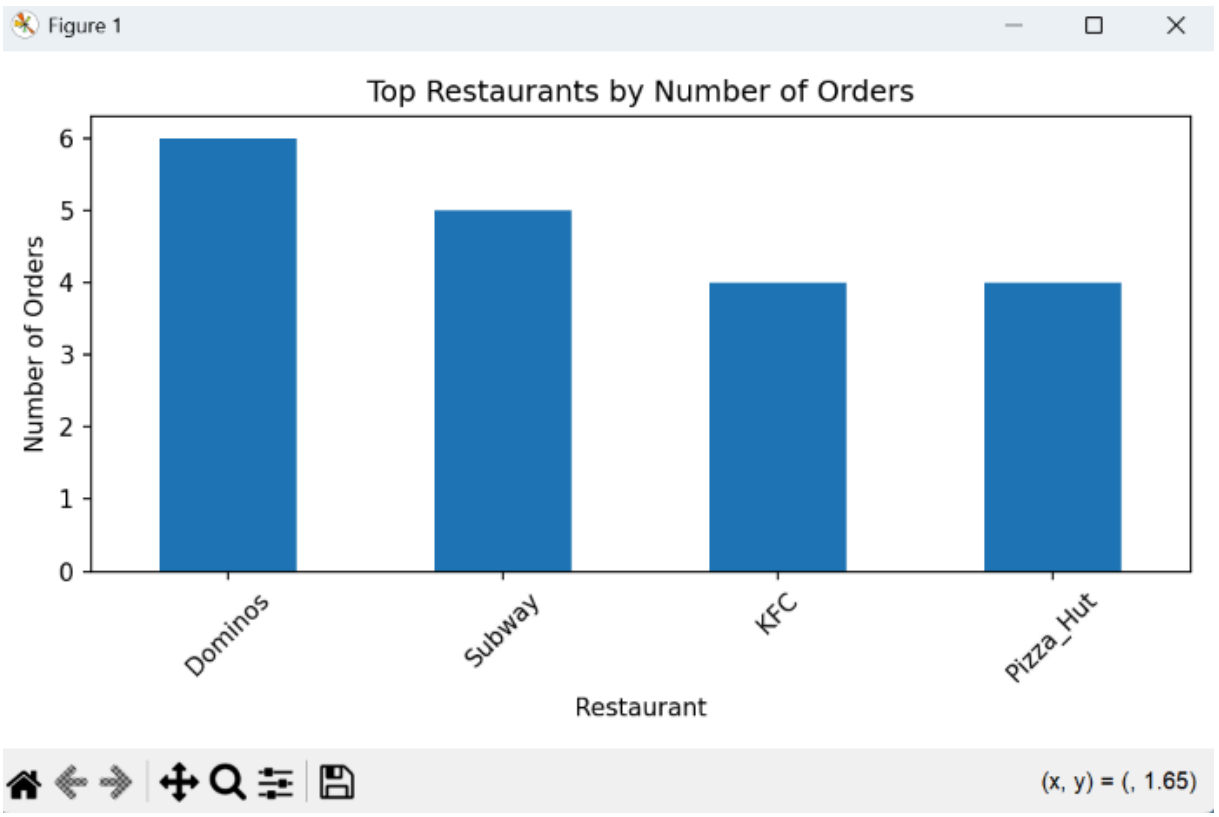
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.histplot(df["Delivery_Time"])
plt.title("Delivery Time Distribution")
plt.show()

df["Restaurant"].value_counts().head(5).plot(kind="bar")
plt.title("Top 5 Restaurants")
plt.show()

sns.countplot(data=df, x="Rating")
plt.title("Customer Ratings")
plt.show()
```


Figure 1
Delivery Time Distribution
Number of Orders
0
1
2
3
4
5
20
25
30
35
40
45
50
55
60
Delivery Time (minutes)


Figure 1
Top Restaurants by Number of Orders
Number of Orders
0
1
2
3
4
5
6
Dominos
Subway
KFC
Pizza_Hut
Restaurant
(x, y) = (, 1.65)



Result: Food delivery data can be cleaned and visualized to identify delays, popular restaurants, and customer satisfaction.

2. Bank Loan Approval Analysis

A. Using Pandas DataFrame operations

Pandas selects important columns such as income, loan amount, credit score, and approval status for analysis.

CODE:

```
import pandas as pd

df = pd.read_csv("loan_data.csv")

selected = df[[
    "Income", "Loan_Amount",
    "Credit_Score", "Approval_Status"
]]
print(selected.head())
```

```
========== RESTART: C:/Users/Madaraboina Gayathri/OneDrive/Desktop/DSA0402-FODS/ASSESSMENTS/C02-AT3/Q2_Bank_
A. ORIGINAL BANK LOAN DATA
    Customer_ID   Income  Loan_Amount  ...  Approval_Status  Gender Employment_Type
0           201  50000.0       200000  ...         Approved    Male        Salaried
1           202  30000.0       150000  ...         Rejected  Female   Self_Employed
2           203  70000.0       300000  ...         Approved    Male        Salaried
3           204  25000.0       100000  ...         Rejected  Female      Unemployed
4           205  60000.0       250000  ...         Approved  Female        Salaried
5           206      NaN       180000  ...         Rejected    Male   Self_Employed
6           207  80000.0       350000  ...         Approved    Male        Salaried
7           208  40000.0       200000  ...         Approved  Female   Self_Employed
8           209  20000.0       120000  ...         Rejected    Male      Unemployed
9           210  55000.0       220000  ...         Approved  Female        Salaried
10          211  35000.0       160000  ...         Rejected    Male   Self_Employed
11          212  90000.0       400000  ...         Approved  Female        Salaried
12          213  45000.0       190000  ...         Approved    Male        Salaried
13          214  28000.0       130000  ...         Rejected  Female      Unemployed
14          215  65000.0       280000  ...         Approved    Male   Self_Employed
15          216  50000.0            0  ...         Rejected  Female        Salaried
16          217  75000.0       320000  ...         Approved    Male        Salaried
17          218  32000.0       140000  ...         Rejected  Female   Self_Employed
18          219  62000.0       260000  ...         Approved  Female        Salaried
19          220  42000.0       210000  ...         Rejected    Male   Self_Employed

[20 rows x 7 columns]
```

```
SELECTED CUSTOMER DETAILS
      Income  Loan_Amount  Credit_Score Approval_Status
0    50000.0       200000           750         Approved
1    30000.0       150000           620         Rejected
2    70000.0       300000           800         Approved
3    25000.0       100000           580         Rejected
4    60000.0       250000           720         Approved
5        NaN       180000           650         Rejected
6    80000.0       350000           820         Approved
7    40000.0       200000           680         Approved
8    20000.0       120000           550         Rejected
9    55000.0       220000           740         Approved
10   35000.0       160000           610         Rejected
11   90000.0       400000           850         Approved
12   45000.0       190000           700         Approved
13   28000.0       130000           590         Rejected
14   65000.0       280000           780         Approved
15   50000.0            0           710         Rejected
16   75000.0       320000           810         Approved
17   32000.0       140000           630         Rejected
18   62000.0       260000           760         Approved
19   42000.0       210000           690         Rejected
```

B. Data preprocessing

- Missing income: Replace missing values with the median income.
- Categorical encoding: Convert gender and employment type into numerical values.
- Normalization: Scale loan amounts between 0 and 1.

CODE:

```
from sklearn.preprocessing import MinMaxScaler

df["Income"] = df["Income"].fillna(df["Income"].median())

df = pd.get_dummies(
    df, columns=["Gender", "Employment_Type"],
    dtype=int
)
```

```
scaler = MinMaxScaler()
df[["Loan_Amount"]] = scaler.fit_transform(
   df[["Loan_Amount"]]
)
```

```
B. PREPROCESSED DATA
    Customer_ID  ...  Employment_Type_Unemployed
0           201  ...                           0
1           202  ...                           0
2           203  ...                           0
3           204  ...                           1
4           205  ...                           0
5           206  ...                           0
6           207  ...                           0
7           208  ...                           0
8           209  ...                           1
9           210  ...                           0
10          211  ...                           0
11          212  ...                           0
12          213  ...                           0
13          214  ...                           1
14          215  ...                           0
16          217  ...                           0
17          218  ...                           0
18          219  ...                           0
19          220  ...                           0
```

C. Data visualization

CODE:

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(data=df, x="Approval_Status", y="Income")
plt.title("Approval Status vs Income")
plt.show()

sns.boxplot(data=df, x="Approval_Status", y="Credit_Score")
plt.title("Approval Status vs Credit Score")
plt.show()

sns.boxplot(data=df, x="Approval_Status", y="Loan_Amount")
plt.title("Approval Status vs Loan Amount")
plt.show()
```

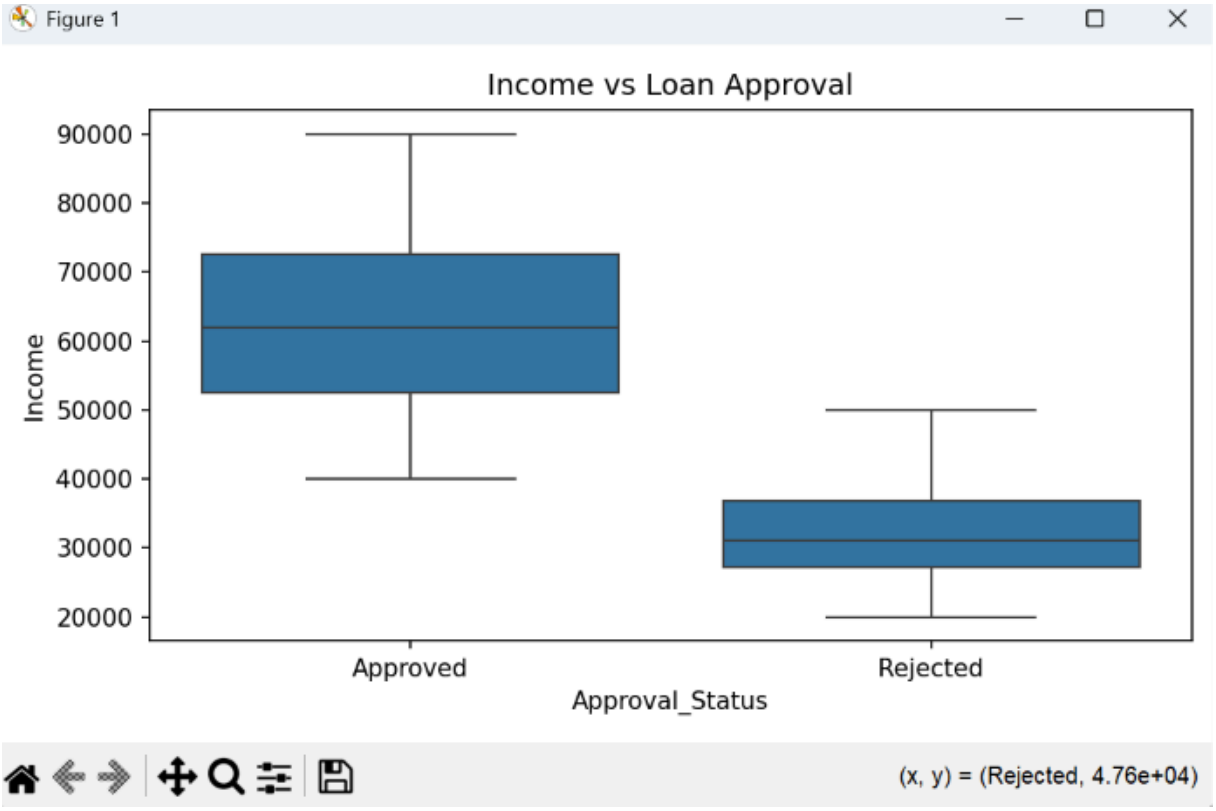
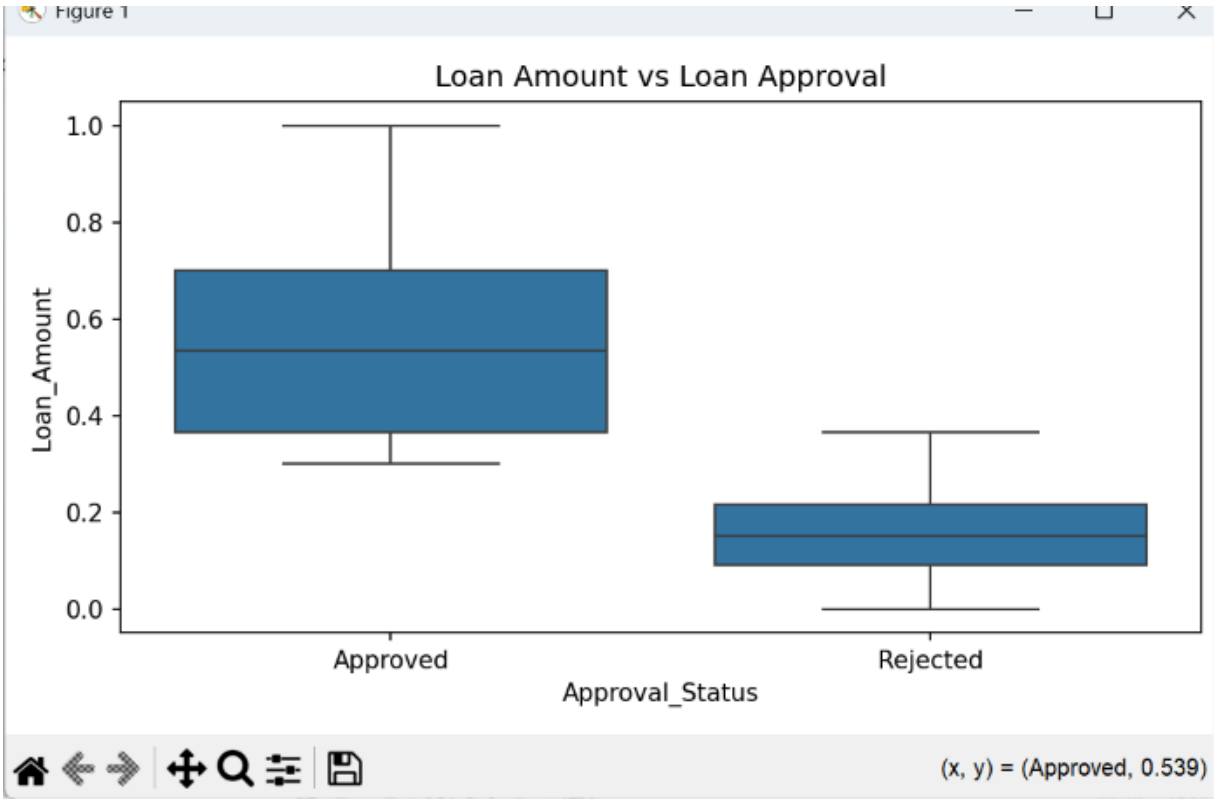

Figure 1
Income vs Loan Approval
Income
90000
80000
70000
60000
50000
40000
30000
20000
Approved
Rejected
Approval_Status
(x, y) = (Rejected, 4.76e+04)


Figure 1
Credit Score vs Loan Approval
Credit_Score
850
800
750
700
650
600
550
Approved
Rejected
Approval_Status
(x, y) = (Approved, 700.1)



Result: Loan application data can be analyzed to understand how income, credit score, and loan amount relate to approval and rejection.

3. Smart Energy Consumption Monitoring

A. Using Pandas to group electricity usage

Pandas groups electricity consumption by appliance and date to calculate total energy usage.

CODE:

```
import pandas as pd


df = pd.read_csv("energy_data.csv")

usage = df.groupby(
    ["Date", "Appliance"]
)["Units"].sum()

print(usage)
```

A. ORIGINAL ENERGY DATA

	Date	Appliance	Units
0	2026-10-01	AC	8.5
1	2026-10-01	Fan	1.2
2	2026-10-01	Light	0.8
3	2026-10-01	Refrigerator	3.0
4	2026-10-02	AC	9.0
5	2026-10-02	Fan	1.5
6	2026-10-02	Light	0.9
7	2026-10-02	Refrigerator	3.1
8	2026-10-03	AC	10.5
9	2026-10-03	Fan	1.3
10	2026-10-03	Light	0.7
11	2026-10-03	Refrigerator	3.0
12	2026-10-04	AC	9.5
13	2026-10-04	Fan	1.4
14	2026-10-04	Light	0.8
15	2026-10-04	Refrigerator	3.2
16	2026-10-05	AC	11.0
17	2026-10-05	Fan	1.6
18	2026-10-05	Light	0.9
19	2026-10-05	Refrigerator	3.1
20	2026-10-06	AC	500.0
21	2026-10-06	Fan	1.4
22	2026-10-06	Light	NaN
23	2026-10-06	Refrigerator	3.0
24	2026-10-07	AC	8.0
25	2026-10-07	Fan	1.2
26	2026-10-07	Light	0.8
27	2026-10-07	Refrigerator	3.3
28	2026-10-08	AC	9.2
29	2026-10-08	Fan	1.5
30	2026-10-08	Light	0.9
31	2026-10-08	Refrigerator	3.1
32	2026-10-09	AC	10.0
33	2026-10-09	Fan	1.4
34	2026-10-09	Light	0.8
35	2026-10-09	Refrigerator	3.2
36	2026-10-10	AC	8.8
37	2026-10-10	Fan	1.3
38	2026-10-10	Light	0.7

ENERGY USAGE BY DATE AND APPLIANCE

Date	Appliance	
2026-10-01	AC	8.5
	Fan	1.2
	Light	0.8
	Refrigerator	3.0
2026-10-02	AC	9.0
	Fan	1.5
	Light	0.9
	Refrigerator	3.1
2026-10-03	AC	10.5
	Fan	1.3
	Light	0.7
	Refrigerator	3.0
2026-10-04	AC	9.5
	Fan	1.4
	Light	0.8
	Refrigerator	3.2
2026-10-05	AC	11.0
	Fan	1.6
	Light	0.9
	Refrigerator	3.1
2026-10-06	AC	500.0
	Fan	1.4
	Light	0.0
	Refrigerator	3.0
2026-10-07	AC	8.0
	Fan	1.2
	Light	0.8
	Refrigerator	3.3
2026-10-08	AC	9.2
	Fan	1.5
	Light	0.9
	Refrigerator	3.1
2026-10-09	AC	10.0
	Fan	1.4
	Light	0.8
	Refrigerator	3.2

B. Data preprocessing

- Remove abnormal readings, such as negative electricity consumption.
- Fill missing consumption values using the median.
- Calculate total power consumption for each appliance and date.

```
df = df[df["Units"] >= 0]

df["Units"] = df["Units"].fillna(
    df["Units"].median()
)

total = df.groupby("Appliance")["Units"].sum()
print(total)
```

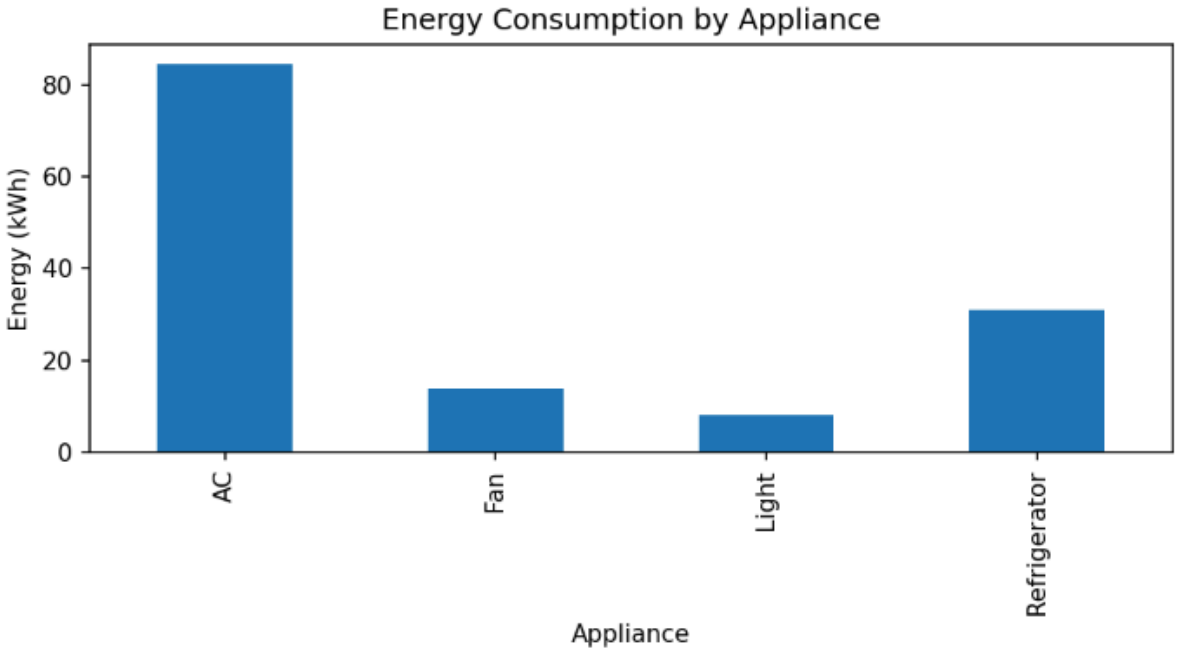
```
B. CLEANED ENERGY DATA
         Date     Appliance  Units
0  2026-10-01            AC    8.5
1  2026-10-01           Fan    1.2
2  2026-10-01         Light    0.8
3  2026-10-01  Refrigerator    3.0
4  2026-10-02            AC    9.0
5  2026-10-02           Fan    1.5
6  2026-10-02         Light    0.9
7  2026-10-02  Refrigerator    3.1
8  2026-10-03            AC   10.5
9  2026-10-03           Fan    1.3
10 2026-10-03         Light    0.7
11 2026-10-03  Refrigerator    3.0
12 2026-10-04            AC    9.5
13 2026-10-04           Fan    1.4
14 2026-10-04         Light    0.8
15 2026-10-04  Refrigerator    3.2
16 2026-10-05            AC   11.0
17 2026-10-05           Fan    1.6
18 2026-10-05         Light    0.9
19 2026-10-05  Refrigerator    3.1
21 2026-10-06           Fan    1.4
22 2026-10-06         Light    0.8
23 2026-10-06  Refrigerator    3.0
24 2026-10-07            AC    8.0
25 2026-10-07           Fan    1.2
26 2026-10-07         Light    0.8
27 2026-10-07  Refrigerator    3.3
28 2026-10-08            AC    9.2
29 2026-10-08           Fan    1.5
30 2026-10-08         Light    0.9
31 2026-10-08  Refrigerator    3.1
32 2026-10-09            AC   10.0
33 2026-10-09           Fan    1.4
34 2026-10-09         Light    0.8
35 2026-10-09  Refrigerator    3.2
36 2026-10-10            AC    8.8
37 2026-10-10           Fan    1.3
```

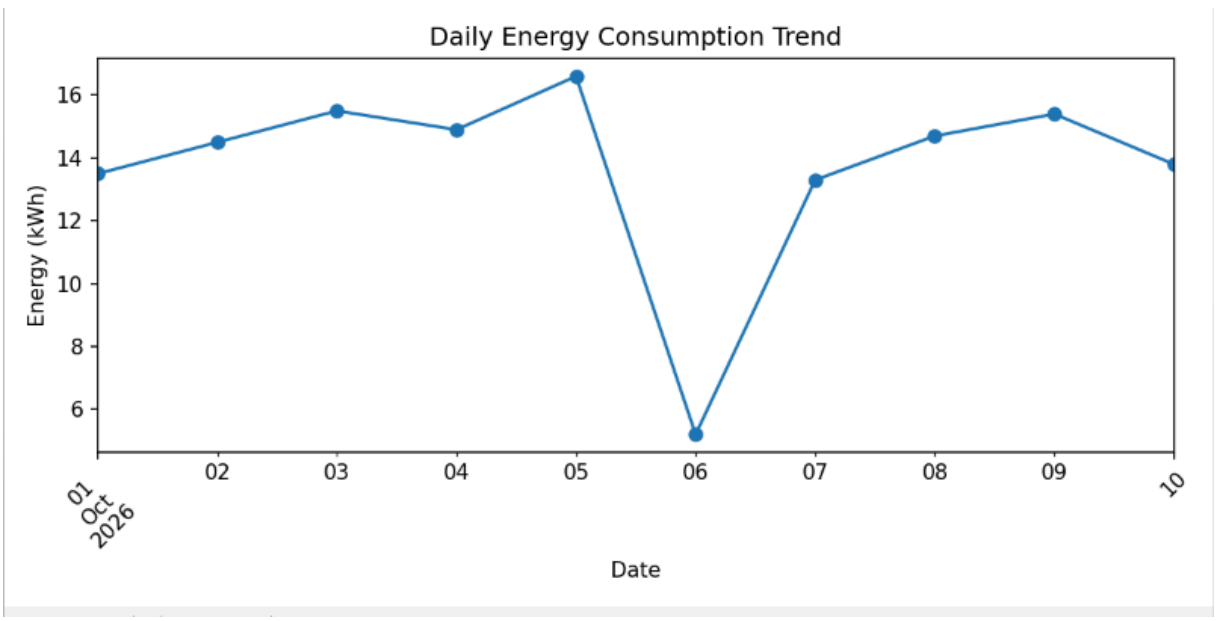
C. Data visualization

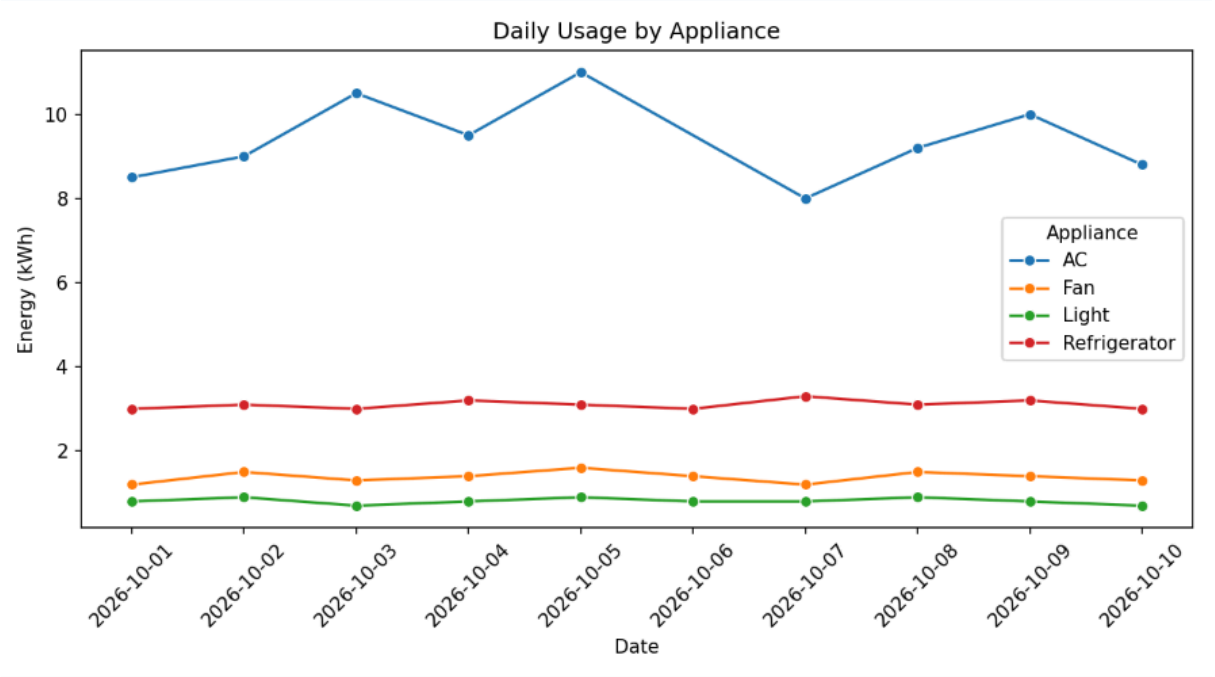
```
import matplotlib.pyplot as plt
```

```
# Bar chart for appliance consumption
total.plot(kind="bar")
plt.title("Energy Consumption by Appliance")
plt.xlabel("Appliance")
plt.ylabel("Units Consumed")
plt.show()

# Line chart for daily energy usage
daily = df.groupby("Date")["Units"].sum()
daily.plot(kind="line", marker="o")
plt.title("Daily Energy Consumption")
plt.xlabel("Date")
plt.ylabel("Units Consumed")
plt.show()
```







Result: Energy consumption analysis identifies appliances that consume the most electricity and reveals daily usage patterns.

4. Online Learning Platform Performance Analysis

A. Using Pandas to sort, filter, and rank students

Pandas can sort students by quiz scores and completion percentage to identify high-performing students.

```
import pandas as pd

df = pd.read_csv("student_data.csv")

sorted_students = df.sort_values(
    by="Quiz_Score", ascending=False
)

df["Rank"] = df["Quiz_Score"].rank(
    ascending=False, method="min"
)

print(sorted_students)
print(df[["Student", "Rank"]])
```

```
STUDENTS SORTED BY QUIZ SCORE
    Student_ID Student  ... Quiz_Score  Assignment_Status
18         319    Tara  ...        96.0          Submitted
3          304   Divya  ...        95.0          Submitted
11         312   Latha  ...        92.0          Submitted
15         316   Priya  ...        90.0          Submitted
0          301    Arun  ...        88.0          Submitted
13         314   Nisha  ...        85.0          Submitted
6          307    Gita  ...        82.0          Submitted
19         320   Vijay  ...        80.0          Submitted
8          309    Isha  ...        78.0          Submitted
4          305    Esha  ...        75.0          Submitted
16         317    Ravi  ...        72.0          Submitted
9          310     Jay  ...        68.0          Submitted
1          302    Bala  ...        65.0          Submitted
12         313   Manoj  ...        55.0            Pending
14         315    Omar  ...        50.0            Pending
2          303  Charan  ...        45.0            Pending
17         318    Sana  ...        40.0            Pending
7          308    Hari  ...        35.0            Pending
10         311   Kiran  ...        25.0            Pending
5          306   Fahad  ...         NaN            Pending

[20 rows x 6 columns]

STUDENTS WITH VIDEO COMPLETION ABOVE 75%
    Student_ID Student  ... Quiz_Score  Assignment_Status
0          301    Arun  ...        88.0          Submitted
3          304   Divya  ...        95.0          Submitted
4          305    Esha  ...        75.0          Submitted
6          307    Gita  ...        82.0          Submitted
8          309    Isha  ...        78.0          Submitted
11         312   Latha  ...        92.0          Submitted
13         314   Nisha  ...        85.0          Submitted
15         316   Priya  ...        90.0          Submitted
16         317    Ravi  ...        72.0          Submitted
18         319    Tara  ...        96.0          Submitted
19         320   Vijay  ...        80.0          Submitted

[11 rows x 6 columns]
```

B. Data preprocessing

- Fill missing quiz scores with the median.
- Convert login dates into a standard date format.
- Create a performance category based on quiz scores.

```
df["Quiz_Score"] = df["Quiz_Score"].fillna(
    df["Quiz_Score"].median()
)

df["Login_Date"] = pd.to_datetime(
    df["Login_Date"], errors="coerce"
)

df["Performance"] = df["Quiz_Score"].apply(
    lambda x: "Excellent" if x >= 80
    else "Average" if x >= 50
```

```
    else "Needs Improvement"
)
```

```
B. PREPROCESSED STUDENT DATA
    Student_ID Student Login_Date  ... Assignment_Status        Performance Rank
0          301    Arun 2026-10-01  ...         Submitted          Excellent    5
1          302    Bala 2026-10-01  ...         Submitted            Average   14
2          303  Charan 2026-10-02  ...           Pending  Needs Improvement   17
3          304   Divya 2026-10-02  ...         Submitted          Excellent    2
4          305    Esha 2026-10-03  ...         Submitted            Average   10
5          306   Fahad 2026-10-03  ...           Pending            Average   10
6          307    Gita 2026-10-04  ...         Submitted          Excellent    7
7          308    Hari 2026-10-04  ...           Pending  Needs Improvement   19
8          309    Isha 2026-10-05  ...         Submitted            Average    9
9          310     Jay 2026-10-05  ...         Submitted            Average   13
10         311   Kiran 2026-10-06  ...           Pending  Needs Improvement   20
11         312   Latha 2026-10-06  ...         Submitted          Excellent    3
12         313   Manoj 2026-10-07  ...           Pending            Average   15
13         314   Nisha 2026-10-07  ...         Submitted          Excellent    6
14         315    Omar 2026-10-08  ...           Pending            Average   16
15         316   Priya 2026-10-08  ...         Submitted          Excellent    4
16         317    Ravi 2026-10-09  ...         Submitted            Average   12
17         318    Sana 2026-10-09  ...           Pending  Needs Improvement   18
18         319    Tara 2026-10-10  ...         Submitted          Excellent    1
19         320   Vijay 2026-10-10  ...         Submitted          Excellent    8

[20 rows x 8 columns]
```

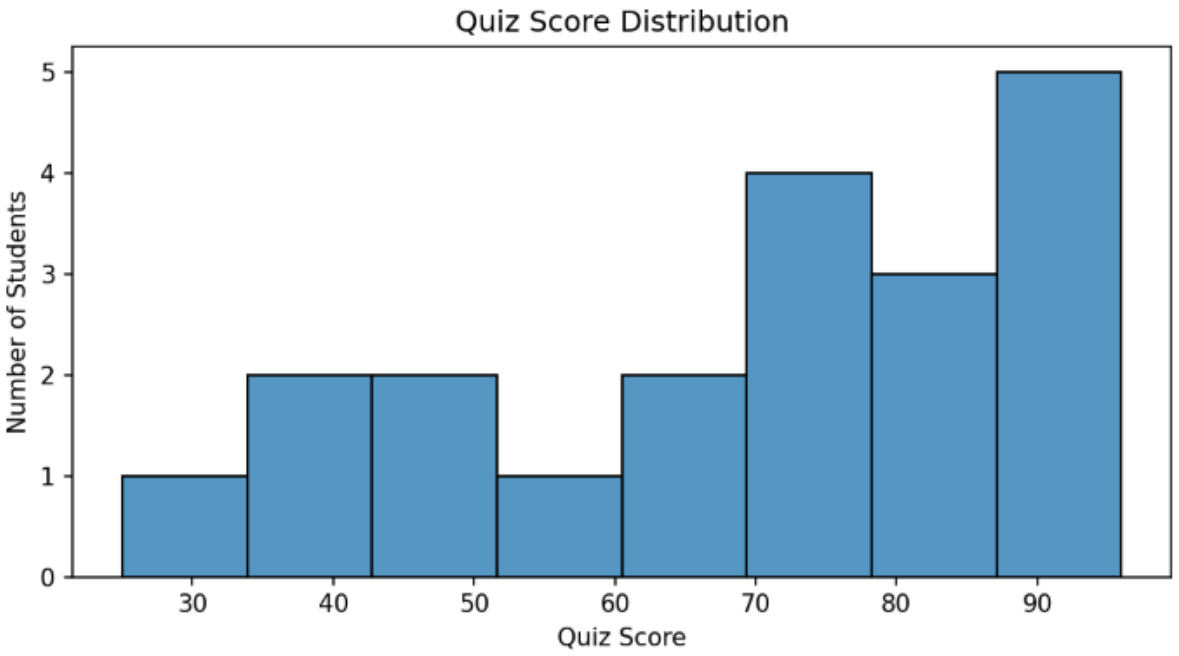
C. Data visualization

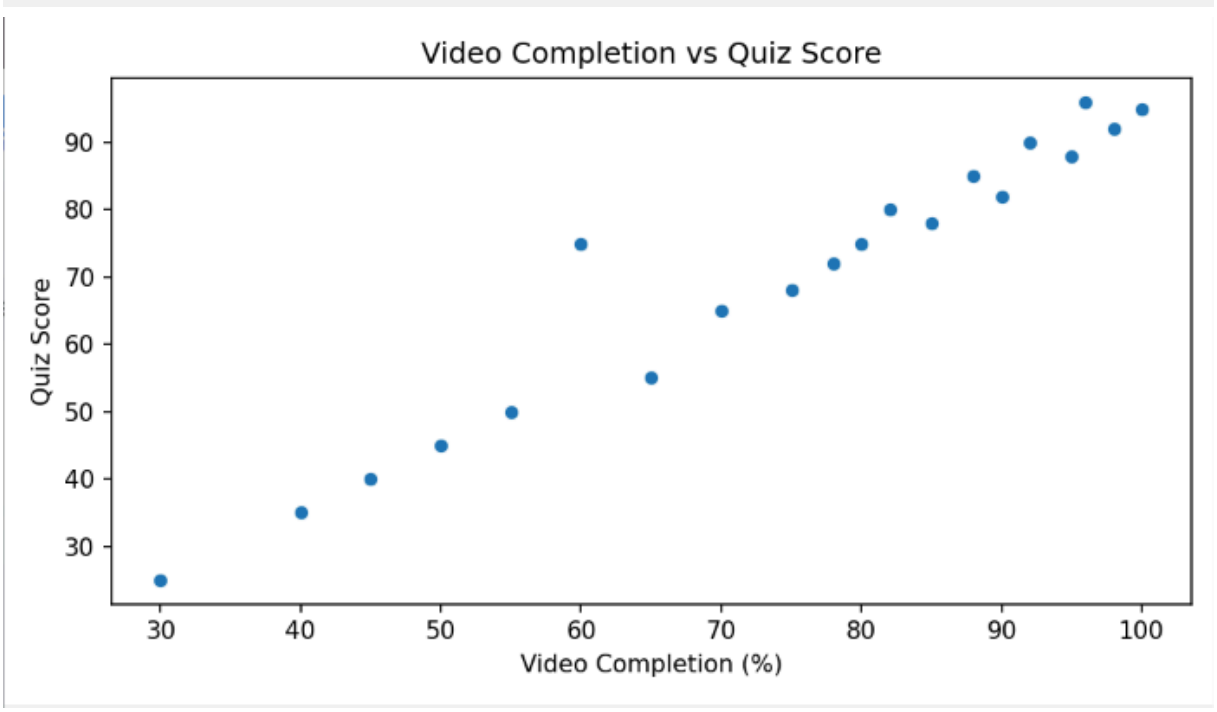
```
import matplotlib.pyplot as plt
import seaborn as sns

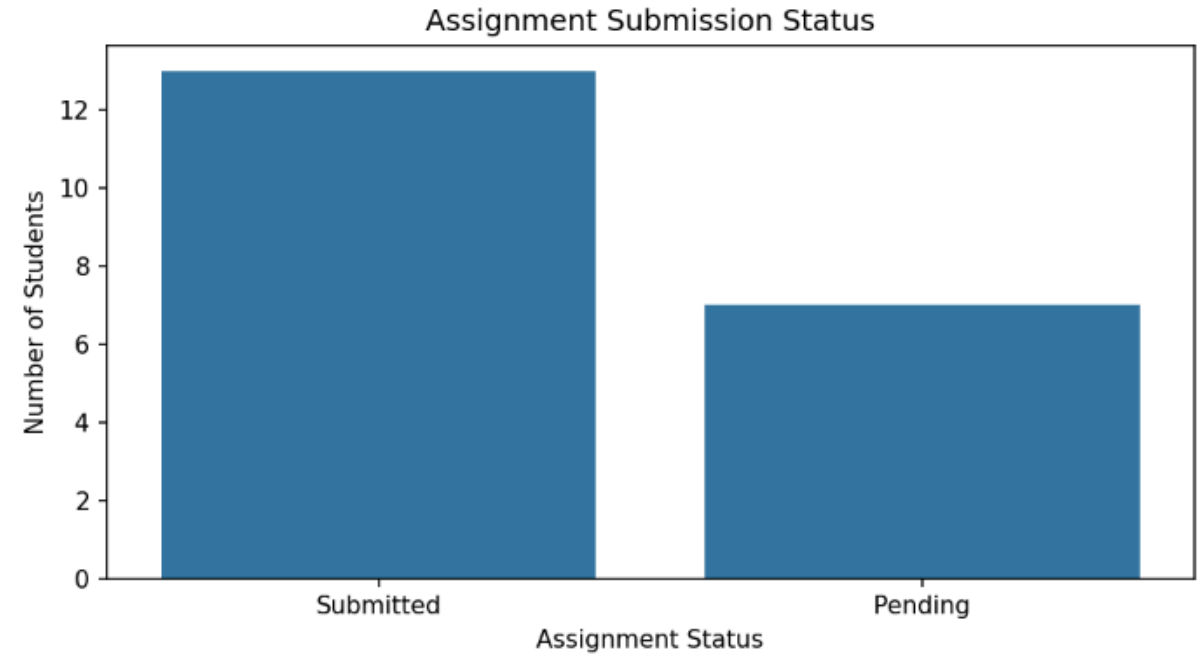
sns.histplot(df["Quiz_Score"], bins=10)
plt.title("Quiz Score Distribution")
plt.show()

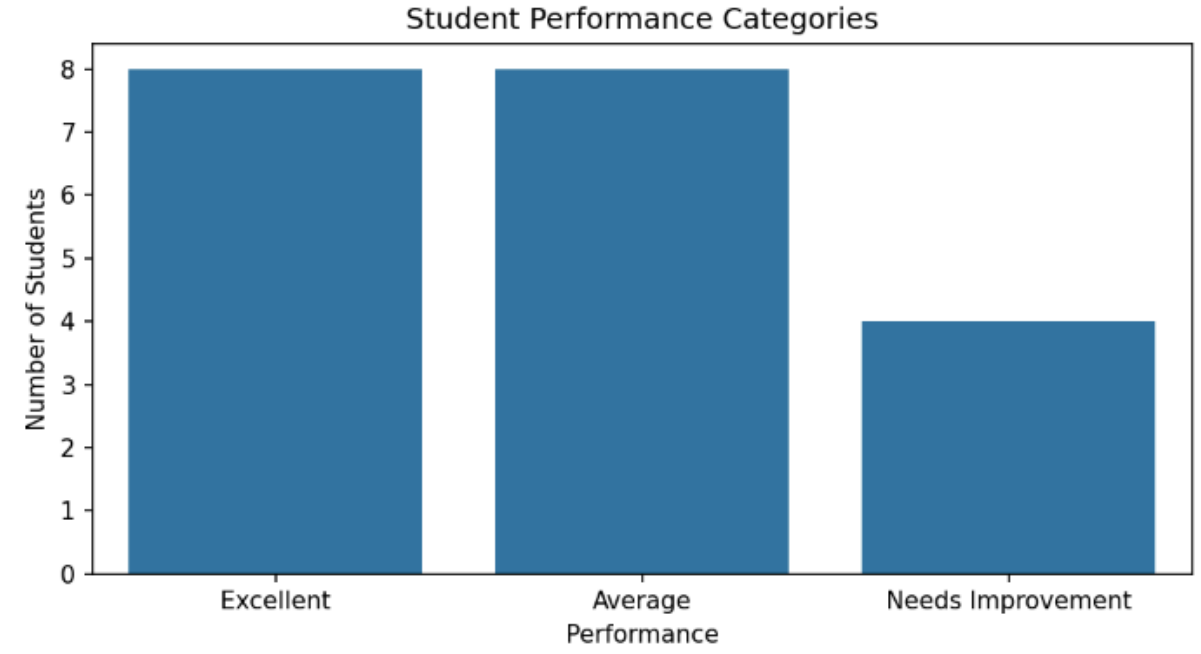
sns.scatterplot(
    data=df,
    x="Completion_Percentage",
    y="Quiz_Score"
)
plt.title("Student Engagement and Performance")
plt.show()

df["Performance"].value_counts().plot(kind="bar")
plt.title("Student Performance Categories")
plt.show()
```


Quiz Score Distribution
Number of Students
0
1
2
3
4
5
30
40
50
60
70
80
90
Quiz Score


Video Completion vs Quiz Score
Quiz Score
30
40
50
60
70
80
90
30
40
50
60
70
80
90
100
Video Completion (%)





Result: Student data analysis helps identify high performers, measure engagement, and determine course completion patterns.

5. Public Transport Passenger Data Analysis

A. Using Pandas to analyze passenger counts

Pandas groups passenger data by route, stop, and time slot to identify busy routes and peak travel hours.

```
import pandas as pd

df = pd.read_csv("bus_passengers.csv")

route_count = df.groupby("Route")["Passengers"].sum()
stop_count = df.groupby("Stop")["Passengers"].sum()
time_count = df.groupby("Time_Slot")["Passengers"].sum()

print(route_count)
print(stop_count)
print(time_count)
```

```
A. ORIGINAL PASSENGER DATA
         Date  Route      Stop Time_Slot  Passengers
0  2026-10-01    101   Central     08:00        45.0
1  2026-10-01    102    Market     08:00        60.0
2  2026-10-01    103   College     09:00        35.0
3  2026-10-01    101   Station     09:00        55.0
4  2026-10-01    102  Hospital     10:00        40.0
5  2026-10-01    103    Market     10:00        30.0
6  2026-10-02    101   Central     08:00        50.0
7  2026-10-02    102    Market     08:00        70.0
8  2026-10-02    103   College     09:00        40.0
9  2026-10-02    101   Station     09:00        65.0
10 2026-10-02    102  Hospital     10:00        45.0
11 2026-10-02    103    Market     10:00        32.0
12 2026-10-03    101   Central     08:00        55.0
13 2026-10-03    102    Market     08:00        75.0
14 2026-10-03    103   College     09:00        38.0
15 2026-10-03    101   Station     09:00        60.0
16 2026-10-03    102  Hospital     10:00         NaN
17 2026-10-03    103    Market     10:00        35.0
18 2026-10-04    101   Central     08:00        48.0
19 2026-10-04    102    Market     08:00        80.0
20 2026-10-04    103   College     09:00        42.0
21 2026-10-04    101   Station     09:00        58.0
22 2026-10-04    102  Hospital     10:00        50.0
23 2026-10-04    103    Market     10:00        33.0
24 2026-10-05    101   Central     08:00        52.0
25 2026-10-05    102    Market     08:00        85.0
26 2026-10-05    103   College     09:00        40.0
27 2026-10-05    101   Station     09:00        62.0
28 2026-10-05    102  Hospital     10:00        47.0
29 2026-10-05    103    Market     10:00        36.0
```

B. Data preprocessing

- Remove records containing invalid route numbers.
- Fill missing passenger counts with the median.
- Remove duplicate passenger records.

```
df["Route"] = pd.to_numeric(
    df["Route"], errors="coerce"
)

df = df.dropna(subset=["Route"])
```

```
df["Route"] = df["Route"].astype(int)

df["Passengers"] = df["Passengers"].fillna(
    df["Passengers"].median()
)

df = df.drop_duplicates()
df = df[df["Passengers"] >= 0]
```

```
B. CLEANED PASSENGER DATA
         Date  Route      Stop Time_Slot  Passengers
0  2026-10-01    101   Central     08:00        45.0
1  2026-10-01    102    Market     08:00        60.0
2  2026-10-01    103   College     09:00        35.0
3  2026-10-01    101   Station     09:00        55.0
4  2026-10-01    102  Hospital     10:00        40.0
5  2026-10-01    103    Market     10:00        30.0
6  2026-10-02    101   Central     08:00        50.0
7  2026-10-02    102    Market     08:00        70.0
8  2026-10-02    103   College     09:00        40.0
9  2026-10-02    101   Station     09:00        65.0
10 2026-10-02    102  Hospital     10:00        45.0
11 2026-10-02    103    Market     10:00        32.0
12 2026-10-03    101   Central     08:00        55.0
13 2026-10-03    102    Market     08:00        75.0
14 2026-10-03    103   College     09:00        38.0
15 2026-10-03    101   Station     09:00        60.0
16 2026-10-03    102  Hospital     10:00        48.0
17 2026-10-03    103    Market     10:00        35.0
18 2026-10-04    101   Central     08:00        48.0
19 2026-10-04    102    Market     08:00        80.0
20 2026-10-04    103   College     09:00        42.0
21 2026-10-04    101   Station     09:00        58.0
22 2026-10-04    102  Hospital     10:00        50.0
23 2026-10-04    103    Market     10:00        33.0
24 2026-10-05    101   Central     08:00        52.0
25 2026-10-05    102    Market     08:00        85.0
26 2026-10-05    103   College     09:00        40.0
27 2026-10-05    101   Station     09:00        62.0
28 2026-10-05    102  Hospital     10:00        47.0
29 2026-10-05    103    Market     10:00        36.0
```

C. Data visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

route_count = df.groupby("Route")["Passengers"].sum()

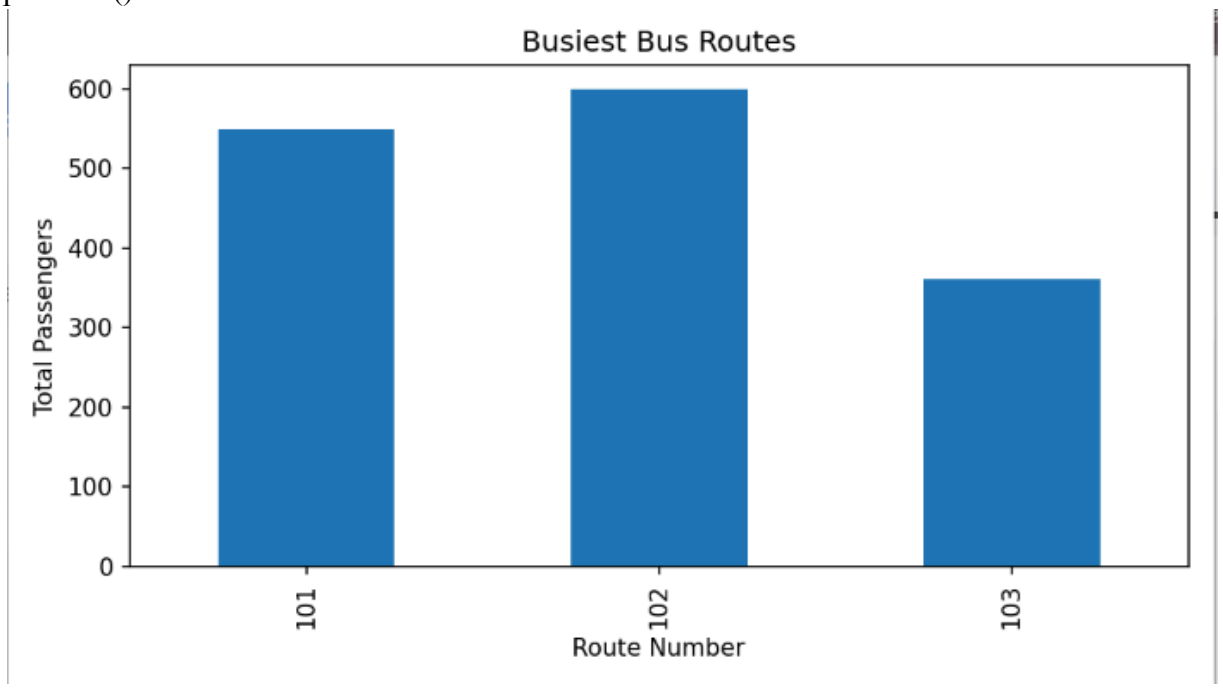
route_count.plot(kind="bar")
plt.title("Busiest Bus Routes")
plt.xlabel("Route")
plt.ylabel("Total Passengers")
plt.show()

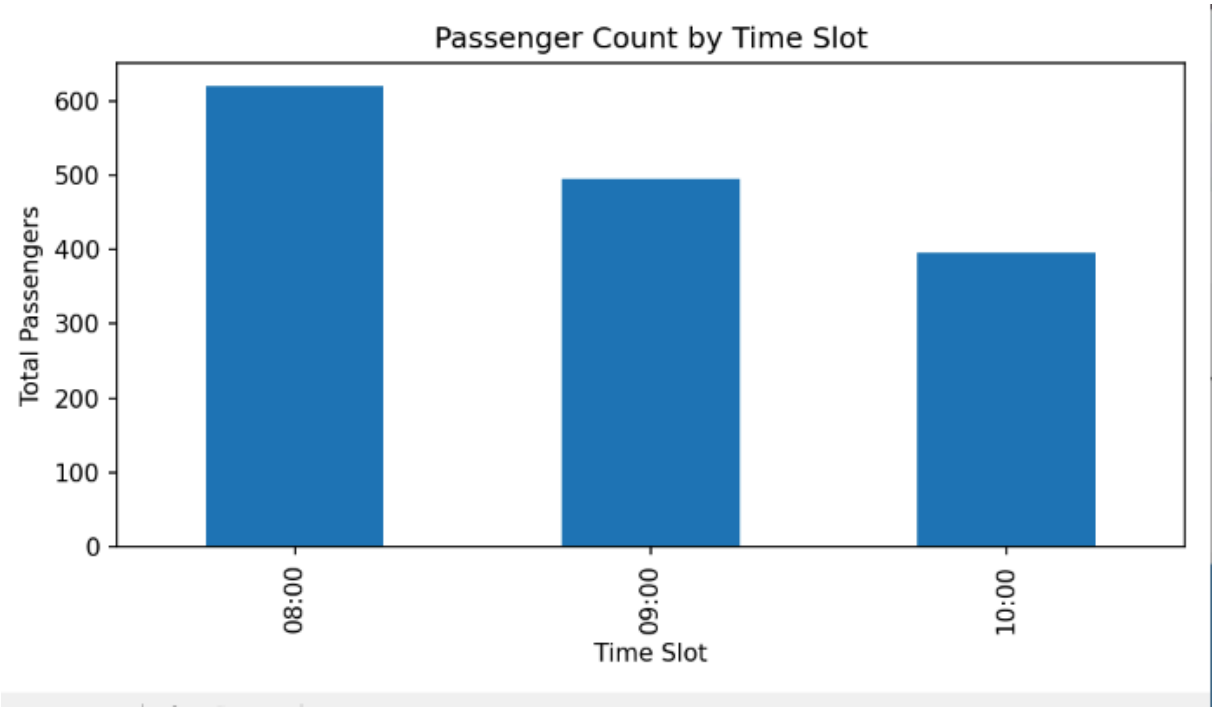
time_count = df.groupby("Time_Slot")["Passengers"].sum()
```

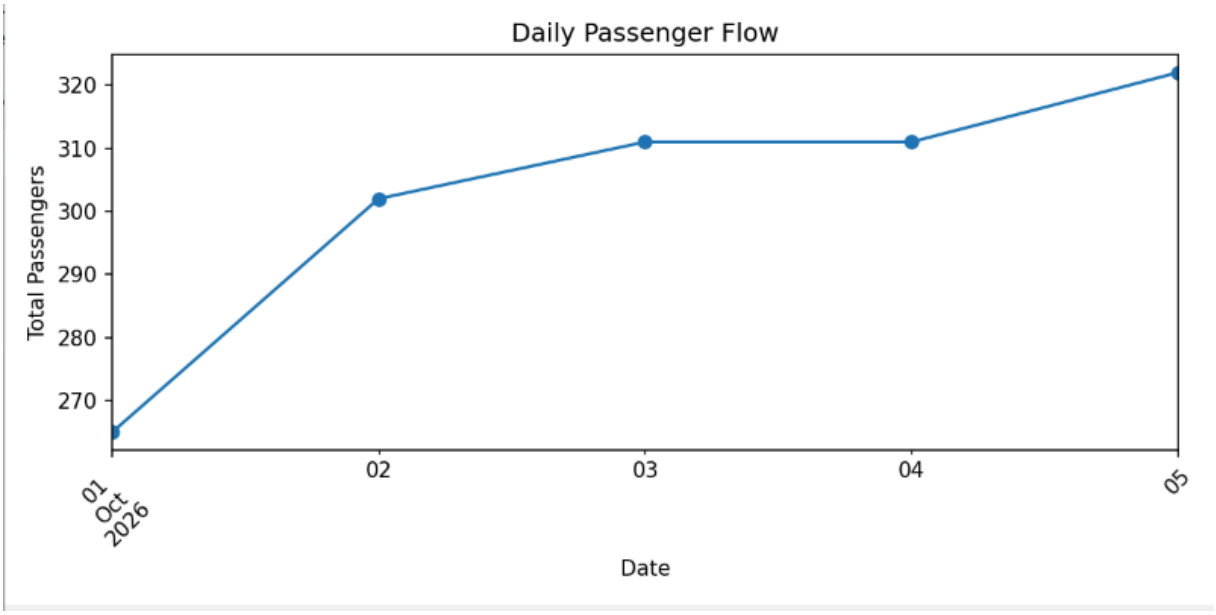
```
sns.barplot(x=time_count.index, y=time_count.values)
plt.title("Peak Travel Hours")
plt.xlabel("Time Slot")
plt.ylabel("Total Passengers")
plt.show()

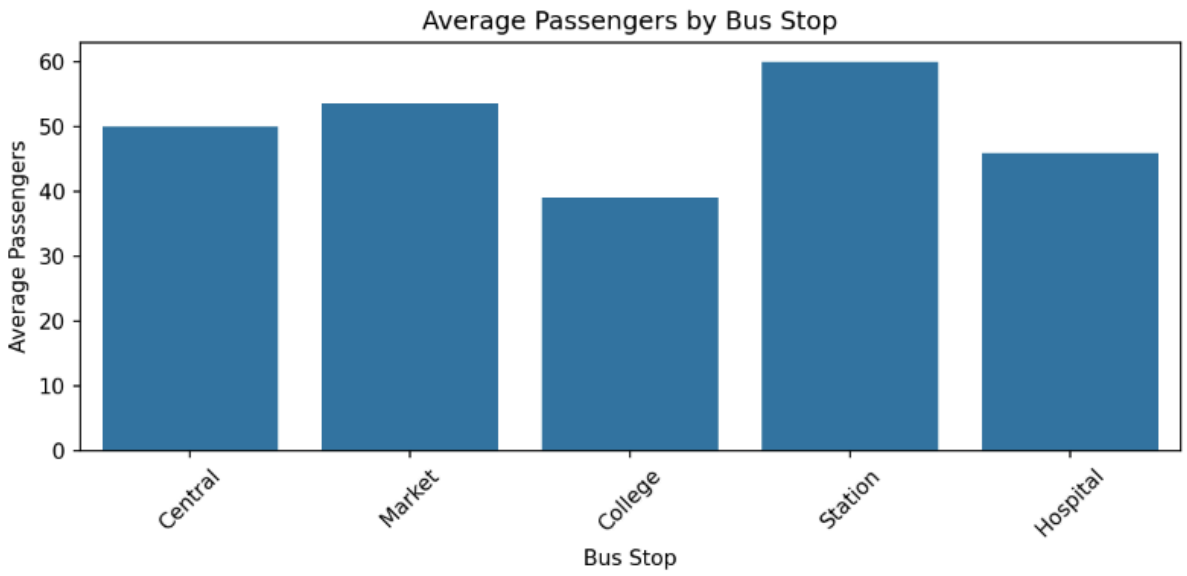
daily_count = df.groupby("Date")["Passengers"].sum()

daily_count.plot(kind="line", marker="o")
plt.title("Daily Passenger Flow")
plt.xlabel("Date")
plt.ylabel("Total Passengers")
plt.show()
```




Passenger Count by Time Slot
Total Passengers
600
500
400
300
200
100
0
08:00
09:00
10:00
Time Slot


Daily Passenger Flow
Total Passengers
320
310
300
290
280
270
01
Oct
2026
02
03
04
05
Date



Result: Passenger data analysis identifies peak travel hours, busiest routes, and passenger flow patterns, helping the transport department improve route planning and reduce overcrowding.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand